

POLITECHNIKA
GDAŃSKA

Narzędzia i Aplikacje Jakarta EE

Jakarta Faces

Michał Wójcik

Jakarta Faces (Java ServerFaces)

Jakarta Faces 2.x - (JSF) (wcześniej Java ServerFaces) oparty na komponentach framework służący do budowania interfejsu użytkownika przeznaczony dla aplikacji webowych działających po stronie serwera:

- jest częścią standardu Java EE od wersji 5,
- umożliwia budowanie interfejsu użytkownika z wykorzystaniem gotowych komponentów,
- obsługuje żądania HTTP:
  - zwraca odpowiedź składającą się z wygenerowanego kodu HTML,
  - przekazuje zdarzenia wygenerowane przez użytkownika (np.: naciśnięcie przycisku) do odpowiednich komponentów;
- posiada model obsługi zdarzań,
- posiada modele dla konwersji i walidacji danych wprowadzanych do komponentów,
- posiada standardowy mechanizm szablonów stron (od JSF 2/Java EE 6),
- posiada wygodny mechanizm relokacji zasobów (np. style CSS, skrypty JavaScript).

Strona JSF

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>
<html lang="en"
      xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">
    <f:view>
        <h:head>
            <meta charset="UTF-8"/>
            <title>Library</title>
        </h:head>
        <h:body>
            <h:outputText value="Hello World"/>
        </h:body>
    </f:view>
</html>
```

Strony JSF to dokumenty XML (zwykle z rozszerzeniem `.xhtml`) łączące znaczniki pochodzące z kilku przestrzeni:

- standardowe komponenty HTML (http://www.w3.org/1999/xhtml),
- komponenty JSF tłumaczone na komponenty HTML (`jakarta.faces.html`, brak formalnego schematu),
- znaczniki konfigurujące zachowanie komponentów (`jakarta.faces.core`, brak formalnego schematu).

Komponenty interfejsu

Komponenty JSF tłumaczone na komponenty HTML (`jakarta.faces.html`):

`inputText`,
`inputSecret`,
`inputTextArea`,
`inputHidden`,
`selectBooleanCheckbox`,
`selectManyCheckbox`,
`selectOneRadio`,
`selectOneListbox`,
`selectManyListbox`,
`selectOneMenu`,
`outputText`,
`outputFormat`,
`graphicImage`,
`outputStylesheet`,
`outputScript`,
`commandButton`,
`link`,
`commandLink`,
`outputLink`,
`panelGrid`,
`message`,
`messages`,
`dataTable`,
`column`.

Konfiguracja komponentów

Znaczniki konfigurujące zachowanie komponentów (`jakarta.faces.core`):

- `param`,
- `attribute`,
- `setPropertyActionListener`,
- `convertNumber`,
- `convertDateTime`,
- `validateLength`,
- `validateLongRange`,
- `validateDoubleRange`,
- `validateRegex`.

JSF servlet

`FacesServlet`:

- zajmuje się obsługą wszystkich żądań do stron JSF,
- może być skonfigurowany w deskryptorze `web.xml`.

```
<servlet>
    <servlet-name>facesServlet</servlet-name>
    <servlet-class>jakarta.faces.webapp.FacesServlet</servlet-class>
</servlet>
```

```
<servlet-mapping>
    <servlet-name>facesServlet</servlet-name>
    <url-pattern>*.xhtml</url-pattern>
</servlet-mapping>
```

Dostarczanie danych do komponentów

Podłączanie beanów do elementów widoku (interfejsu użytkownika) może być zrealizowane jako:

- podłączenie wartości komponentu do właściwości beana poprzez Expression Language (EL),
- podłączenie instancji komponentu do pola beana (binding):
  - nie konwertuje automatycznie wartości wpisanych w komponencie,
  - beany stają się zależne od klas JSF,
  - pozwalają z poziomu beana sterować komponentem (widzialność, wygląd itd.).

```
<h:body>
    <h:outputText value="#{helloBean.helloText"/>
</h:body>
```

Expression Language

Expression Language - język wyrażeń udostępniający dwa typy odwołań:

- `${wyrazenie}` - natychmiastowa ewaluacja:
  - tylko do odczytu (rvalue),
  - obliczane po pierwszym wyświetleniu strony (immediate),
  - szeroko używane na stronach JSP;
- `#{wyrazenie}`:
  - do odczytu i zapisu (rvalue, lvalue),
  - obliczane w dowolnym etapie cyklu życia strony (deferred),
  - szeroko używane w widokach JSF.

Można go używać:

- w statycznym tekście oraz w atrybucie znacznika, który na to pozwala,
- wyrażenie jest automatycznie konwertowane do potrzebnego typu (`String`, `int`, ...).

EL - dostęp do zmiennych

Dostęp do **zmiennych**:

- `${variableName}`,
- kontener będzie szukał zmiennej o podanej nazwie kolejno w kontekstach:
  - strony,
  - żądania,
  - sesji,
  - aplikacji.

EL - dostęp do właściwości

Dostęp do **właściwości** obiektów następuje przez:

- `.` - znak kropki:
  - `${person.name}`;
- `[]`:
  - `${person['name']}`.

Dla wyrażania `${a.b}` lub `${a['b']}`:

- jeśli zmienna jest mapą zwróci obiekt pod kluczem `b`,
- jeśli zmienna jest beanem zwraca `a.getB()`.

Dla wyrażania `${a[b]}`:

- jeśli zmienna jest listą zwraca obiekt spod indeksu `b`.

EL - operatory

Operatory:

- operatory arytmetyczne:
  - `+`,
  - `-`,
  - `*`,
  - `/`, `div`,
  - `%`, `mod`;
- operatory logiczne:
  - `and`, `&&`,
  - `or`, `||`,
  - `not`, `!`,
  - `==`, `eq`,
  - `!=`, `ne`,
  - `<`, `lt`,
  - `>`, `gt`,
  - `<=`, `le`,
  - `>=`, `ge`;
- operator `empty`:
  - `empty`,
- operator warunkowy:
  - `A ? B : C`.

EL - predefiniowane zmienne

Są **predefiniowane** następujące **zmienne**:

- `pageContext` - kontekst strony JSP, zawiera w sobie:
  - `servletContext`,
  - `session`,
  - `request`,
  - `response`,
- `session`,
- `request`,
- `param`,
- `paramValues`,
- `header`,
- `headerValues`,
- `cookie`,
- `initParam`,
- `pageScope` - zasięg strony JSP,
- `viewScope` - zasięg widoku JSF,
- `requestScope`,
- `sessionScope`,
- `applicationScope`.

Zasięgi

Obiekty backing beans można umieszczać w różnych zasięgach:

- aplikacji,
- sesji,
- żądania,
- widoku,
- żaden - nowa instancja przy każdym użyciu;
- własny (programisty).

Konfiguracja zasięgu:

- Java EE 5, 6:
  - beany zarządzane JSF, adnotacje z pakietu `java.faces.bean`,
  - `@CustomScoped`, `@NoneScoped`, `@ViewScoped`, `@RequestScoped`, `@SessionScoped`, `@ApplicationScoped`,
  - tymczasowe rozwiązanie w oczekiwaniu na specyfikację CDI;
- Java EE 7+, Jakarta EE 9+:
  - beany CDI,
  - `@Dependent`, `@ViewScoped`, `@RequestScoped`, `@ConversationScoped`, `@SessionScoped`, `@ApplicationScoped`,

Bean JSF

```
import javax.faces.bean.ManagedBean;
import javax.faces.bean.RequestScoped;

@ManagedBean
@RequestScoped
public class BookList implements Serializable {

    private List<Book> books;

    public List<Book> getBooks() {
        if (book == null) {
            //fetch from data source
        }
        return books;
    }

}
```

Bean CDI

```
import jakarta.enterprise.context.RequestScoped;
import jakarta.inject.Named;

@Named
@RequestScoped
public class BookList implements Serializable {

    private List<Book> books;

    public List<Book> getBooks() {
        if (book == null) {
            //fetch from data source
        }
        return books;
    }

}
```

Strona JSF - model danych

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>
<html lang="en"
      xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:f="jakarta.faces.core">
    <f:view>
        <h:head>
            <meta charset="UTF-8"/>
            <title>Library</title>
        </h:head>
        <h:body>
            <h:outputText value="Hello World"/>
            <h:dataTable value="#{bookList.books}" var="book">
                <h:column>
                    <h:outputText value="#{book.title}"/>
                </h:column>
            </h:dataTable>
        </h:body>
    </f:view>
</html>
```

Konwertery, walidatory, detektory

Konwertery:

- konwertują wartości wpisywane w formularzach na typy języka Java,
- standardowo dostępne są konwertery do wszystkich podstawowych typów (w tym dat),
- definiowanie własnych poprzez implementację interfejsu `Converter`,
- rejestrowanie własnych poprzez `@FacesConverter`.

Walidatory:

- odpowiadają za kontrolę wpisywanych wartości w pola formularzy,
- można samemu zaimplementować walidację jako metodę w beanie lub klasę spełniającą interfejs `Validator`.

Detektory zdarzeń:

- pozwalają nasłuchiwać na pewne zdarzenie jak: kliknięcie przycisku lub odnośnika, zmiana wartości pola itp.,
- implementuje się je jako metodę w beanie lub klasę spełniającą odpowiedni interfejs (`ValueChangeListener`, `ActionListener`).

Deskryptor faces-config.xml

Deskryptor `faces-config.xml` - nieobowiązkowy plik konfiguracyjny, w którym można definiować m. in.:

- konfigurację zasobów (np. z tłumaczeniami),
- nawigację pomiędzy widokami,
- beany zarządzane, konwertery, walidatory, detektory zdarzeń,
- konfigurację obsługi elementów frameworku JSF, np. własną obsługę widoków (`ViewHandler`).

Możliwa konfiguracja za pomocą adnotacji (dłuższy czas wdrażania aplikacji – konieczność skanowania klas).

Wywołanie akcji

Wykonanie nawigacji:

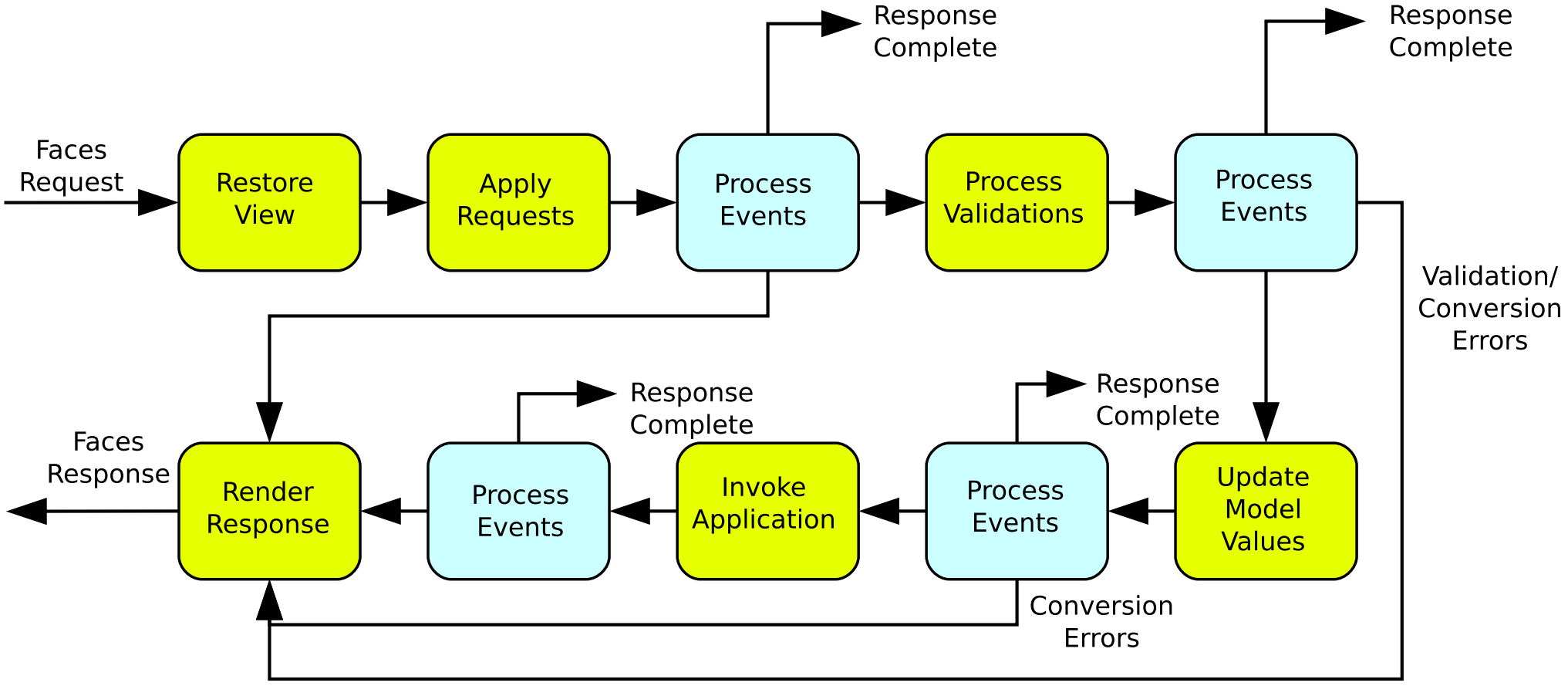
```
<h:button value="Books" outcome="book_list.xhtml"/>
```

Wywołanie metody z backing bean z nawigacją:

```
<h:form>
    <h:outputLabel value="Title:" for="title"/>
    <h:inputText id="title" value="#{bookEdit.book.title}"/>
    <h:message for="title"/>

    <h:commandButton value="Save" action="#{bookEdit.saveBook}"/>
    <h:messages/>
</h:form>
```

Żądanie JSF


Faces Request
Restore View
Apply Requests
Process Events
Response Complete
Process Validations
Process Events
Response Complete
Validation/ Conversion Errors
Update Model Values
Process Events
Response Complete
Conversion Errors
Invoke Application
Process Events
Response Complete
Render Response
Faces Response

Żądanie JSF - fazy

Fazy żądania JSF:

- Restore View - tworzenie widoku strony, tworzenie instancji dla każdego komponentu, wiązanie walidatorów, konwerterów i procedur obsługi zdarzeń z komponentami,
- Apply Requests - skonwertowanie i przypisanie wartości z żądania do komponentów,
- Process Validations - walidacja wartości w komponentach,
- Update Model - przypisanie wartości z komponentów do modelu,
- Invoke Applications-obsługa zdarzeń na poziomie aplikacji, zgłaszanie formularza, linkowanie do innych stron,
- Render Response - generowanie odpowiedzi.

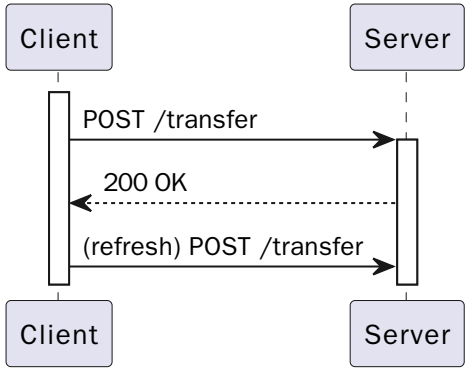
Wielokrotne przesyłanie formularzy

Wielokrotne przesyłanie formularzy:

- deweloper korzystający z JSF przez większość czasu nie przejmuje się protokołem HTTP,
- problem wielokrotnego przesłania formularza jest obecny w JSF tak, jak w każdym innym frameworku.

POLITECHNIKA GDAŃSKA

Problem wielokrotnego zatwierdzania

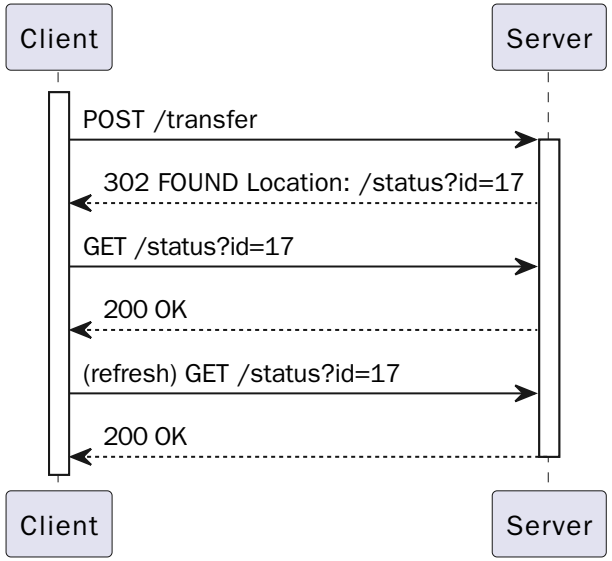


POST i przycisk Odśwież

Wielokrotne przesyłanie formularzy:

- przeglądarka *pamięta* ostatnio wysłane żądanie,
- odświeżenie strony spowoduje ponowne wysłanie identycznego żądania,
- współczesne przeglądarki ostrzegają przed ponownym wysłaniem zapytania POST,
- niepotrzebny i potencjalnie niezrozumiały komunikat dla użytkownika.

Idiom POST-REDIRECT-GET



POST i zmiany stanu

Poprawny przepływ:

- żądanie POST, które zmieniło stan po stronie serwera, nie powinno zwracać informacji dla użytkownika, a jedynie odpowiedź Redirect (grupa 3xx) dla przeglądarki,
  - przeglądarka zrealizuje przekierowanie pod adres umieszczony w nagłówku `Location`,
  - docelowa strona powinna prezentować status wykonanej operacji;
- jeśli nie nastąpiła zmiana stanu, przekierowanie nie jest konieczne:
  - np. błędnie wypełniony formularz.

Redirect w JSF

Parametr `faces-redirect` umożliwia wysłanie odpowiedzi 302 do przeglądarki, np.

```
public String saveBook() {
    bookService.saveBook(book);
    return"list_books?faces-redirect=true";
}
```

```
<h:form>
    <h:outputLabel value="Title:" for="title"/>
    <h:inputText id="title" value="#{bookEdit.book.title}"/>
    <h:message for="title"/>

    <h:commandButton value="Save" action="#{bookEdit.saveBook}"/>
    <h:messages/>
</h:form>
```

Facelets

Facelets:

- zalecany język deklaracji widoków w JSF 2,
- widok deklaruje się jako standardowy plik XHTML z przestrzeniami nazw JSF, Facelets i JSTL,
- pozwala na tworzenie szablonów stron w prosty sposób (znaczniki `<ui:insert>` i `<ui:define>`,
- pozwala na łatwe tworzenie komponentów złożonych,
- wspiera wykorzystanie EL.

Facelets - wspierane biblioteki

Wspierane biblioteki:

- JavaServer Faces Facelets Tag Library - tworzenie szablonów,
- JavaServer Faces HTML Tag Library - renderowanie komponentów HTML,
- JavaServer Faces Core Tag Library - bazowe akcje niezależne od komponentów renderujących (render kit),
- JSTL Functions Tag Library - funkcje z JSTL.

Szablony - definicja

Definicja szablonu:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>
<html lang="en"
      xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html"
      xmlns:ui="jakarta.faces.facelets"
      xmlns:f="jakarta.faces.core">
    <f:view>
        <h:head>
            <meta charset="UTF-8"/>
            <title>Library</title>
        </h:head>
        <h:body>
            <section>
                <ui:insert name="content"/>
            </section>
        </h:body>
    </f:view>
</html>
```

Szablony - wykorzystanie

Strona korzystająca z szablonu:

```
<?xml version="1.0" encoding="UTF-8"?>
<ui:composition xmlns="http://www.w3.org/1999/xhtml"
                xmlns:h="jakarta.faces.html"
                xmlns:ui="jakarta.faces.facelets"
                xmlns:f="jakarta.faces.core"
                template="/WEB-INF/template/main.xhtml">

    <ui:define name="content">
        <h:dataTable value="#{bookList.books}" var="book">
        <h:column>
            <h:outputText value="#{book.title}"/>
        </h:column>
    </ui:define>

</ui:composition>
```

Pomocne źródła

Specyfikacja:

- https://jakarta.ee/specifications/faces/4.0/jakarta-faces-4.0
- https://jakarta.ee/specifications/expression-language/5.0/jakarta-expression-language-spec-5.0

Uzupełnienie informacji:

- https://eclipse-ee4j.github.io/jakartaee-tutorial/#jakarta-faces-technology-2
- https://www.baeldung.com/spring-jsf
- https://www.baeldung.com/jsf-spring-boot-controller-service-dao
- https://www.baeldung.com/jsf-expression-language-el-3
- https://www.baeldung.com/intro-to-jsf-expression-language